

Elevate Fitness App

Backend-Focused User Stories & Acceptance Criteria

For Backend Engineering Team

Epic 1: Authentication & Identity Service

User Story 1.1 — User Registration

Endpoint: POST /api/v1/auth/register

Role: Public (Anonymous)

Description: Register a new athlete account and flag it for profile completion.

Request Payload

```
{ firstName, lastName, email, password, phoneNumber }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid payload; email unique; password ≥ 6 chars with 1 uppercase + 1 number; phone matches Egyptian format	POST /auth/register	Hash password, insert row into Users, return 201 with userId and requiresProfileCompletion: true.
Email already registered	POST /auth/register	Return 409 Conflict — AUTH_EMAIL_EXISTS.
Required field missing, or password/phone fails format rules	POST /auth/register	Return 400 Bad Request — VAL_REQUIRED_FIELD or AUTH_WEAK_PASSWORD.

 **Backend Notes:** Tables: Users (Insert). Never log raw passwords — store bcrypt hash only. firstName/lastName: 2–50 chars. Password: min 6 chars, 1 uppercase, 1 number.

User Story 1.2 — Complete Profile

Endpoint: POST /api/v1/auth/complete-profile

Role: Authenticated

Description: Close out the registration flow and initialize downstream profile records.

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Authenticated user with requiresProfileCompletion = true	POST /auth/complete-profile	Flip the completion flag on Users, emit an internal event so the Profile Service creates UserProfiles / UserPreferences rows, return 200.
Token missing or invalid	POST /auth/complete-profile	Return 401 Unauthorized — AUTH_TOKEN_INVALID.

 **Backend Notes:** Tables: Users (Update). Triggers UserProfiles / UserPreferences initialization inside the User Profile Service.

User Story 1.3 — User Login & Account Lockout

Endpoint: POST /api/v1/auth/login

Role: Public (Anonymous)

Request Payload

```
{ email, password }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid credentials, account not locked	POST /auth/login	Log a success row in LoginAttempts, return 200 with token, refreshToken, profileCompleted, isPremium.
Invalid email or password	POST /auth/login	Log a failed row in LoginAttempts, return 401 — AUTH_INVALID_CREDENTIALS.
5 failed attempts within a 15-minute window	POST /auth/login	Set IsLockedOut = true, compute LockedUntil = now + 15 minutes, return 423 — AUTH_ACCOUNT_LOCKED.
Login attempted while still inside an active lockout window	POST /auth/login	Reject immediately with 423 — AUTH_ACCOUNT_LOCKED without re-checking the password.

 **Backend Notes:** Tables: Users (Read/Update), LoginAttempts (Insert). Lockout rule: 5 failures / 15 minutes.

User Story 1.4 — Forgot Password (Request OTP)

Endpoint: POST /api/v1/auth/forgot-password

Role: Public (Anonymous)

Request Payload

```
{ email }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Registered email submitted	POST /auth/forgot-password	Generate a 6-digit code, store its hash in OtpCodes with ExpiresAt = now + 10 minutes, return 200 with otpExpiresIn: 600, canResendIn: 30.
Resend requested before the 30-second cooldown elapses	POST /auth/forgot-password	Return 429 — RATE_OTP_RESEND_TOO_SOON. Do not regenerate the code.

 **Backend Notes:** Tables: OtpCodes (Insert). Code is hashed before storage. TTL: 10 minutes. Resend cooldown: 30 seconds.

User Story 1.5 — Verify OTP

Endpoint: POST /api/v1/auth/verify-otp

Role: Public (Anonymous)

Request Payload

```
{ email, otp }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Correct OTP submitted within the expiry window	POST /auth/verify-otp	Mark the OtpCodes row IsUsed = true, issue a short-lived resetToken, return 200.
Incorrect OTP submitted	POST /auth/verify-otp	Return 400 — AUTH_INVALID_OTP.
OTP expired (> 10 minutes old)	POST /auth/verify-otp	Return 400 — AUTH_OTP_EXPIRED. Do not issue a resetToken.

 **Backend Notes:** Tables: OtpCodes (Update). resetToken is single-use with a short TTL.

User Story 1.6 — Reset Password

Endpoint: POST /api/v1/auth/reset-password

Role: Public (Anonymous, holding a resetToken)

Request Payload

```
{ resetToken, newPassword, confirmPassword }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid resetToken; newPassword matches confirmPassword and meets policy	POST /auth/reset-password	Update Users.PasswordHash, invalidate the resetToken/OtpCodes row, return 200 with passwordChanged: true.
newPassword does not match confirmPassword	POST /auth/reset-password	Return 400 — AUTH_PASSWORD_MISMATCH.
resetToken invalid, expired, or already used	POST /auth/reset-password	Return 400 — AUTH_RESET_TOKEN_INVALID.

 **Backend Notes:** Tables: Users (Update PasswordHash), OtpCodes (Update IsUsed). Revoke all active RefreshTokens for the user on a successful reset.

User Story 1.7 — Refresh Token

Endpoint: POST /api/v1/auth/refresh-token

Role: Public (carries a valid refresh token)

Request Payload

```
{ refreshToken }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Token exists in RefreshTokens, not revoked, not expired	POST /auth/refresh-token	Revoke the old token (RevokedAt = now), insert a new token + refresh pair, return 200 with the new token/refreshToken.
Token not found or already revoked	POST /auth/refresh-token	Return 401 — AUTH_TOKEN_INVALID.
Token expired	POST /auth/refresh-token	Return 401 — AUTH_TOKEN_EXPIRED.

 **Backend Notes:** Tables: RefreshTokens (Update/Insert). Tokens rotate on every use — the old token is revoked immediately.

User Story 1.8 — Logout

Endpoint: POST /api/v1/auth/logout

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid bearer token supplied	POST /auth/logout	Revoke the caller's active RefreshTokens row(s), return 200 with loggedOut: true.
Token missing or invalid	POST /auth/logout	Return 401 — AUTH_TOKEN_INVALID.

 **Backend Notes:** Tables: RefreshTokens (Update RevokedAt).



Epic 2: User Profile & Settings Service

User Story 2.1 — View Profile

Endpoint: GET /api/v1/profile

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid token	GET /profile	Return UserProfiles fields merged with a cached statistics snapshot (totalWorkouts, currentStreak, etc.), 200.
Token missing or invalid	GET /profile	Return 401 — AUTH_TOKEN_INVALID.
No profile row exists for the user	GET /profile	Return 404 — RES_NOT_FOUND.

 *Backend Notes: Tables: UserProfiles (Read), UserStatistics (Read, merged into the response). IsPremiumCached is synced from Subscription Service events, not queried live.*

User Story 2.2 — Update Profile

Endpoint: PUT /api/v1/profile

Role: Authenticated

Request Payload

```
{ firstName, lastName, email, phoneNumber }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid payload; email not already used by a different account	PUT /profile	Update the UserProfiles row, return 200.
Email already used by another account	PUT /profile	Return 409 — AUTH_EMAIL_EXISTS.
Required field missing	PUT /profile	Return 400 — VAL_REQUIRED_FIELD.

 *Backend Notes: Tables: UserProfiles (Update).*

User Story 2.3 — Upload Profile Picture

Endpoint: POST /api/v1/profile/picture

Role: Authenticated

Request Payload

Multipart form-data — field profilePicture (binary file)

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
File is JPG or PNG and ≤ 5MB	POST /profile/picture	Store the file, update UserProfiles.ProfilePictureUrl, return 200.
File format is not JPG/PNG	POST /profile/picture	Return 400 — VAL_INVALID_FILE_TYPE.
File exceeds 5MB	POST /profile/picture	Return 400 — VAL_FILE_TOO_LARGE.
Storage write fails	POST /profile/picture	Return 500 — SRV_FILE_UPLOAD_FAILED.

 *Backend Notes: Tables: UserProfiles (Update ProfilePictureUrl). Max size 5MB; formats restricted to JPG/PNG.*

User Story 2.4 — Change Password

Endpoint: PUT /api/v1/profile/change-password

Role: Authenticated

Request Payload

```
{ currentPassword, newPassword, confirmPassword }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
currentPassword correct; newPassword matches confirmPassword and meets policy	PUT /profile/change-password	Update Users.PasswordHash, return 200.
currentPassword incorrect	PUT /profile/change-password	Return 401 — AUTH_INVALID_CREDENTIALS.
newPassword does not match confirmPassword	PUT /profile/change-password	Return 400 — AUTH_PASSWORD_MISMATCH.

 *Backend Notes: Tables: Users (Update PasswordHash). Consider revoking other active refresh tokens on success.*

User Story 2.5 — View Settings

Endpoint: GET /api/v1/settings

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid token	GET /settings	Return a grouped JSON view of UserPreferences, NotificationSettings, and PrivacySettings, 200.
Token missing or invalid	GET /settings	Return 401 — AUTH_TOKEN_INVALID.

 *Backend Notes: Tables: UserPreferences, NotificationSettings, PrivacySettings (Read, matched on UserId).*

User Story 2.6 — Update Settings (Partial Patch)

Endpoint: PUT /api/v1/settings

Role: Authenticated

Request Payload

```
{ theme, notifications: { workoutReminders } } (partial — any subset of keys)
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Partial JSON payload containing one or more valid keys	PUT /settings	Patch only the supplied fields across UserPreferences / NotificationSettings / PrivacySettings, return 200. Untouched fields are left exactly as they were.

Given	When	Then (Expected Backend Behavior)
Unknown key or invalid value supplied	PUT /settings	Return 400 — VAL_REQUIRED_FIELD.

 **Backend Notes:** Tables: *UserPreferences, NotificationSettings, PrivacySettings (Update)*. Must behave as a true PATCH, never a full-object replace.

Epic 3: Fitness Calculation Engine (FCE)

User Story 3.1 — Submit Weight, Goal & Activity Stats

Endpoint: POST /api/v1/fitness/weight-goal-activity

Role: Authenticated

Request Payload

```
{ userId, weight, height, age, gender, goal, activityLevel }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Age 16–100, Weight 40–200kg, Height 140–220cm; gender/goal/activityLevel match the allowed enums	POST /fitness/weight-goal-activity	Insert a new row into UserFitnessStats with RecordedAt = now, return 200/201.
Age, weight, or height outside the allowed range	POST /fitness/weight-goal-activity	Return 400 — VAL_INVALID_AGE / VAL_INVALID_WEIGHT / VAL_INVALID_HEIGHT.
Gender not Male or Female	POST /fitness/weight-goal-activity	Return 400 — VAL_INVALID_GENDER.
Goal or activityLevel not in the allowed enum list	POST /fitness/weight-goal-activity	Return 400 — VAL_INVALID_GOAL / VAL_INVALID_ACTIVITY.

 **Backend Notes:** Tables: *UserFitnessStats (Insert)*. Goal enum: *Lose Weight, Get Fitter, Gain Weight, Gain More Flexible, Learn the Basic*. ActivityLevel enum: *Rookie, Beginner, Intermediate, Advance, TrueBeast*.

User Story 3.2 — Calculate BMR, TDEE & Calorie Target

Endpoint: POST /api/v1/fitness/calculate

Role: Authenticated

Request Payload

```
{ userId }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
An active UserFitnessStats row exists for userId	POST /fitness/calculate	Apply the gender-specific BMR formula, multiply by the activity factor for TDEE, apply

Given	When	Then (Expected Backend Behavior)
		the goal offset for calorieTarget, classify Status, upsert CalculatedMetrics, return 200 with bmr / tdee / calorieTarget / status.
No UserFitnessStats row has been recorded yet	POST /fitness/calculate	Return 404 — FCE_STATS_NOT_FOUND.
Calculation produces a non-finite or invalid numeric result	POST /fitness/calculate	Return 400 — FCE_INVALID_CALCULATION.

📌 *Backend Notes: Male BMR = $10 \times \text{weight}(\text{kg}) + 6.25 \times \text{height}(\text{cm}) - 5 \times \text{age} + 5$. Female BMR = $10 \times \text{weight} + 6.25 \times \text{height} - 5 \times \text{age} - 161$. TDEE = BMR $\times$ activity factor (Rookie 1.2, Beginner 1.375, Intermediate 1.55, Advance 1.725, TrueBeast 1.9). Calorie Target = TDEE adjusted by goal (Lose Weight -500, Gain Weight +300, Gain More Flexible +150, Get Fitter/Learn the Basic = TDEE). Status: ≤ 1800 Weak, 1801–2500 Normal, > 2500 Hard. Tables: CalculatedMetrics (Upsert by UserId).*

User Story 3.3 — Assign Fitness Plan

Endpoint: POST /api/v1/fitness/assign-plan

Role: Authenticated

Request Payload

```
{ userId }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
CalculatedMetrics exist; a FitnessPlanConfig row matches Goal + Status	POST /fitness/assign-plan	Set the prior UserAssignedPlans row IsActive = false, insert a new active assignment, append a snapshot to UserPlanHistory, return 200 with the plan detail.
No CalculatedMetrics row exists yet	POST /fitness/assign-plan	Return 400 — FCE_METRICS_NOT_CALCULATED.
No FitnessPlanConfig row matches the Goal + Status combination	POST /fitness/assign-plan	Return 404 — FCE_NO_MATCHING_PLAN.

📌 *Backend Notes: Tables: UserAssignedPlans (Update/Insert), UserPlanHistory (Insert), FitnessPlanConfig (Read).*

User Story 3.4 — Get Fitness Metrics

Endpoint: GET /api/v1/fitness/metrics/{userId}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
A CalculatedMetrics row exists for userId	GET /fitness/metrics/{userId}	Return bmr / tdee / calorieTarget / status / calculatedAt, 200.
No metrics have been calculated yet	GET /fitness/metrics/{userId}	Return 400 — FCE_METRICS_NOT_CALCULATED.

 Backend Notes: Tables: CalculatedMetrics (Read).

User Story 3.5 — Get Fitness Stats

Endpoint: GET /api/v1/fitness/stats/{userId}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
A UserFitnessStats row exists for userId	GET /fitness/stats/{userId}	Return weight / height / age / gender / goal / activityLevel, 200.
No stats have been recorded for userId	GET /fitness/stats/{userId}	Return 404 — FCE_STATS_NOT_FOUND.

 Backend Notes: Tables: UserFitnessStats (Read latest row by RecordedAt).

User Story 3.6 — Recalculate Metrics & Reassign Plan

Endpoint: PUT /api/v1/fitness/recalculate/{userId}

Role: Authenticated (also invoked internally via the weight_updated event)

Request Payload

```
{ reason, newWeight, triggeredBy } (optional)
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Triggered by the weight_updated event or a manual newWeight submission	PUT /fitness/recalculate/{userId}	Re-run the BMR/TDEE/calorieTarget formulas, overwrite CalculatedMetrics, compare previousStatus to newStatus.
Recalculation yields a different Status than before	PUT /fitness/recalculate/{userId}	Re-run plan reassignment: deactivate the current UserAssignedPlans row, insert a new one, append to UserPlanHistory, return planReassignment: true.
Recalculation yields the same Status as before	PUT /fitness/recalculate/{userId}	Update CalculatedMetrics only; return planReassignment: false, with no new UserAssignedPlans/UserPlanHistory rows.
userId has no prior stats or metrics	PUT /fitness/recalculate/{userId}	Return 400 — FCE_METRICS_NOT_CALCULATED.

 *Backend Notes: Tables: CalculatedMetrics (Update), UserAssignedPlans (Update/Insert if reassigned), UserPlanHistory (Insert if reassigned). Called synchronously or via the async weight_updated broker event from the Progress Tracking Service.*

User Story 3.7 — Get Fitness Plan Configurations

Endpoint: GET /api/v1/fitness/plan-configs

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid optional filters: goal, status, page, pageSize	GET /fitness/plan-configs	Return a paginated list of FitnessPlanConfig rows matching the filters, 200.
No rows match the supplied filters	GET /fitness/plan-configs	Return 200 with an empty array, never 404.

 *Backend Notes: Tables: FitnessPlanConfig (Read). Defaults: page = 1, pageSize = 20.*

User Story 3.8 — Get Specific Plan Configuration

Endpoint: GET /api/v1/fitness/plans/{planId}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
planId exists in FitnessPlanConfig	GET /fitness/plans/{planId}	Return full plan detail (duration, workoutsPerWeek, programType, calorie range), 200.
planId not found	GET /fitness/plans/{planId}	Return 404 — RES_PLAN_NOT_FOUND.

 *Backend Notes: Tables: FitnessPlanConfig (Read).*



Epic 4: Workout & Exercise Catalog Service

User Story 4.1 — Browse Workouts

Endpoint: GET /api/v1/workouts

Role: Authenticated

Request Payload

Query: page, pageSize, category, difficulty, duration, search

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid query parameters supplied	GET /workouts	Join Workouts with WorkoutPlans, apply filters, sort by OrderIndex ASC, return a paginated list, 200.
No workouts match the supplied filters	GET /workouts	Return 200 with an empty array.
category value is not a recognized enum	GET /workouts?category=	Return 400 — VAL_REQUIRED_FIELD (invalid filter value).

 *Backend Notes: Tables: Workouts, WorkoutPlans (Read, indexed joins). category enum: full-body, chest, arms, shoulders, back, legs, stomach.*

User Story 4.2 — Get Workout Details

Endpoint: GET /api/v1/workouts/{id}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
id exists	GET /workouts/{id}	Return the workout joined with its WorkoutExercises, ordered by OrderIndex ASC, 200.
id not found	GET /workouts/{id}	Return 404 — RES_WORKOUT_NOT_FOUND.

 *Backend Notes: Tables: Workouts, WorkoutExercises, Exercises (Read joins).*

User Story 4.3 — Get Workouts by Plan

Endpoint: GET /api/v1/workouts/by-plan/{planId}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
planId exists	GET /workouts/by-plan/{planId}	Return all Workouts rows with that PlanId, sorted by OrderIndex, 200.
planId not found	GET /workouts/by-plan/{planId}	Return 404 — RES_PLAN_NOT_FOUND.

 *Backend Notes: Tables: Workouts (Read, filtered by PlanId).*

User Story 4.4 — Get Workouts by Category

Endpoint: GET /api/v1/workouts/category/{categoryName}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
categoryName is a valid enum value	GET /workouts/category/{categoryName}	Return matching Workouts rows, 200.
categoryName is not a recognized category	GET /workouts/category/{categoryName}	Return 404 — RES_NOT_FOUND.

 Backend Notes: Tables: Workouts (Read, Category index).

User Story 4.5 — Start Workout Session

Endpoint: POST /api/v1/workouts/{id}/start

Role: Authenticated

Request Payload

```
{ difficulty, plannedDuration }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Workout id exists	POST /workouts/{id}/start	Insert a WorkoutSessions row with Status = 'Active', return 201 with sessionId and the ordered exercise list.
Workout id not found	POST /workouts/{id}/start	Return 404 — RES_WORKOUT_NOT_FOUND.
Caller already has an Active session for this workout	POST /workouts/{id}/start	Return the existing sessionId rather than creating a duplicate row (idempotent start).

 Backend Notes: Tables: WorkoutSessions (Insert). SessionId is a globally unique token.

User Story 4.6 — Browse Exercise Library

Endpoint: GET /api/v1/exercises

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid request	GET /exercises	Return a paginated list of Exercises, 200.

 Backend Notes: Tables: Exercises (Read).

User Story 4.7 — Get Exercise Detail

Endpoint: GET /api/v1/exercises/{id}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
id exists	GET /exercises/{id}	Return name, targetMuscles, equipment, description, videoUrl, 200.
id not found	GET /exercises/{id}	Return 404 — RES_EXERCISE_NOT_FOUND.

 *Backend Notes: Tables: Exercises (Read).*

User Story 4.8 — Browse Workout Plans

Endpoint: GET /api/v1/workout-plans

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid request	GET /workout-plans	Return a paginated list of WorkoutPlans, 200.

 *Backend Notes: Tables: WorkoutPlans (Read).*

User Story 4.9 — Get Workout Plan Detail

Endpoint: GET /api/v1/workout-plans/{planId}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
planId exists	GET /workout-plans/{planId}	Return plan detail (Goal, Status, Difficulty, Description), 200.
planId not found	GET /workout-plans/{planId}	Return 404 — RES_PLAN_NOT_FOUND.

 *Backend Notes: Tables: WorkoutPlans (Read).*



Epic 5: Progress Tracking Service

User Story 5.1 — Log Workout Completion

Endpoint: POST /api/v1/progress/workouts

Role: Authenticated

Request Payload

```
{ workoutId, sessionId, completedAt, duration, caloriesBurned, difficulty, notes, rating, exercisesCompleted[] }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
sessionId references a valid Active WorkoutSessions row owned by the caller	POST /progress/workouts	Insert a WorkoutLogs row, bulk-insert WorkoutLogExercises, transition WorkoutSessions.Status to 'Completed', increment Streaks/UserStatistics, evaluate achievements, return 201 with logId / streakUpdated / currentStreak.
sessionId not found or does not belong to the caller	POST /progress/workouts	Return 404 — RES_SESSION_NOT_FOUND.
rating outside 1–5, or a required field missing	POST /progress/workouts	Return 400 — VAL_REQUIRED_FIELD.
Logging this workout newly satisfies an achievement threshold	POST /progress/workouts	Insert UserAchievements row and publish an achievement_earned event to the Notification Service.

 **Backend Notes:** Tables: WorkoutSessions (Update), WorkoutLogs (Insert), WorkoutLogExercises (multi-row Insert), Streaks (Counter Update), UserStatistics (Update Accumulators), UserAchievements (Insert on unlock).

User Story 5.2 — Log Weight Entry

Endpoint: POST /api/v1/progress/weight

Role: Authenticated

Request Payload

```
{ weight, date, notes }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
weight within 40–200kg	POST /progress/weight	Insert a WeightHistory row, publish an asynchronous weight_updated event targeting the FCE, return 201 with bmi / differenceFromPrevious / totalWeightLost.
weight outside 40–200kg	POST /progress/weight	Return 400 — VAL_INVALID_WEIGHT.
date is not a valid date format	POST /progress/weight	Return 400 — VAL_INVALID_DATE.

 **Backend Notes:** Tables: WeightHistory (Insert), UserStatistics (Update). Async Message Broker Hook: weight_updated event → FCE Service triggers PUT /fitness/recalculate/{userId} automatically.

User Story 5.3 — View Progress Dashboard

Endpoint: GET /api/v1/progress

Role: Authenticated

Request Payload

Query: period (week/month/year/all), startDate, endDate

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid token, optional period filter	GET /progress	Aggregate summary stats, weight history, workout history, week-over-week comparisons, and earned achievements scoped to the period, return 200.
startDate is after endDate	GET /progress	Return 400 — VAL_INVALID_DATE.

 *Backend Notes: Tables: WorkoutLogs, WeightHistory, UserStatistics, UserAchievements (Read, aggregated).*

User Story 5.4 — View User Progress by Id

Endpoint: GET /api/v1/progress/{userId}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
userId exists and the caller is authorized to view it	GET /progress/{userId}	Return the same aggregated progress payload scoped to that user, 200.
userId not found	GET /progress/{userId}	Return 404 — RES_USER_NOT_FOUND.

 *Backend Notes: Tables: WorkoutLogs, WeightHistory, UserStatistics (Read).*

User Story 5.5 — View Weight History

Endpoint: GET /api/v1/progress/weight-history/{userId}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Rows exist in WeightHistory for userId	GET /progress/weight-history/{userId}	Return the chronological list of weight entries, 200.
No entries recorded yet	GET /progress/weight-history/{userId}	Return 200 with an empty array.

 Backend Notes: Tables: WeightHistory (Read, sorted by Date).

User Story 5.6 — View Achievements

Endpoint: GET /api/v1/progress/achievements

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Caller has earned achievements	GET /progress/achievements	Return UserAchievements joined with the Achievements master rows (name, description, imageUrl, earnedAt), 200.
No achievements earned yet	GET /progress/achievements	Return 200 with an empty array.

 Backend Notes: Tables: Achievements, UserAchievements (Read joins).

User Story 5.7 — View Progress Stats

Endpoint: GET /api/v1/progress/stats/{userId}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
A UserStatistics row exists for userId	GET /progress/stats/{userId}	Return totalWorkouts / totalCaloriesBurned / totalWeightLost / currentStreak / longestStreak, 200.
userId not found	GET /progress/stats/{userId}	Return 404 — RES_USER_NOT_FOUND.

 Backend Notes: Tables: UserStatistics, Streaks (Read).

Epic 6: Nutrition Service

User Story 6.1 — Get Personalized Meal Recommendations

Endpoint: GET /api/v1/nutrition/recommendations

Role: Authenticated

Request Payload

Query: mealType, page, pageSize, maxCalories, minProtein

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Caller already has a calculated CalorieTarget in the FCE	GET /nutrition/recommendations	Synchronously call the FCE to read the active CalorieTarget, match Meals rows by Type and caloric range, return userDailyGoalCalories + recommendedMeals, 200.
CalorieTarget not yet calculated for the caller	GET /nutrition/recommendations	Return 400 — FCE_METRICS_NOT_CALCULATED.
No meals match the supplied filters	GET /nutrition/recommendations	Return 200 with an empty recommendedMeals array.

 *Backend Notes: Tables: Meals (Read). Synchronous REST call: Nutrition Service → FCE Service to extract the live CalorieTarget.*

User Story 6.2 — Get Meal Recommendations by User Id

Endpoint: GET /api/v1/nutrition/recommendations/{userId}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
userId has calculated metrics	GET /nutrition/recommendations/{userId}	Apply the same matching logic scoped explicitly to userId, 200.
userId has no calculated metrics	GET /nutrition/recommendations/{userId}	Return 400 — FCE_METRICS_NOT_CALCULATED.

 *Backend Notes: Tables: Meals (Read), FCE CalculatedMetrics (Read via internal service call).*

User Story 6.3 — Get Meal Detail

Endpoint: GET /api/v1/nutrition/meals/{id}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
id exists	GET /nutrition/meals/{id}	Return nutrition facts, ingredients, instructions, variations, allergens, and tags, 200.
id not found	GET /nutrition/meals/{id}	Return 404 — RES_MEAL_NOT_FOUND.

 *Backend Notes: Tables: Meals (Read).*

User Story 6.4 — Browse Meal Plans

Endpoint: GET /api/v1/nutrition/meal-plans

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Valid request	GET /nutrition/meal-plans	Return a paginated MealPlans list with its MealPlanItems schedule, 200.

 *Backend Notes: Tables: MealPlans, MealPlanItems (Read joins).*

User Story 6.5 — Get Meal Plans by Calorie Range

Endpoint: GET /api/v1/nutrition/meal-plans/by-calories

Role: Authenticated

Request Payload

Query: calories

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
calories parameter supplied	GET /nutrition/meal-plans/by-calories?calories=	Return MealPlans rows where TargetCalorieRangeMin/Max bracket the supplied value, 200.
calories parameter missing	GET /nutrition/meal-plans/by-calories	Return 400 — VAL_REQUIRED_FIELD.

 *Backend Notes: Tables: MealPlans (Read, range filter).*

Epic 7: Smart Coach & Home Feed Service (Low Priority)

User Story 7.1 — Send Chat Message

Endpoint: POST /api/v1/smart-coach/chat

Role: Authenticated

Request Payload

```
{ message, sessionId }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
sessionId valid, or omitted to start a new session	POST /smart-coach/chat	Pull user context from the FCE and Progress Service, feed it into the AI prompt, store User and AI rows in ChatMessages, return reply + followUpSuggestions, 200.

Given	When	Then (Expected Backend Behavior)
Free-tier caller has exceeded 5 messages in 24 hours	POST /smart-coach/chat	Return 403 — PERM_PREMIUM_REQUIRED.
sessionId supplied but not found	POST /smart-coach/chat	Return 404 — RES_SESSION_NOT_FOUND.

 **Backend Notes:** Tables: ChatSessions (Insert if new), ChatMessages (Insert × 2 per turn). Sender enum: User, AI. Free tier capped at 5 messages / 24h via a cache check.

User Story 7.2 — Get Chat History

Endpoint: GET /api/v1/smart-coach/history

Role: Authenticated

Request Payload

Query: page, pageSize, sessionId (optional)

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
sessionId supplied and owned by the caller	GET /smart-coach/history?sessionId=	Return the ordered ChatMessages for that session, 200.
No sessionId supplied	GET /smart-coach/history	Return a paginated list of the caller's ChatSessions, 200.

 **Backend Notes:** Tables: ChatSessions, ChatMessages (Read).

User Story 7.3 — Get Home Feed

Endpoint: GET /api/v1/home

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
RecommendationCache row exists and CurrentTime < ExpiresAt	GET /home	Return the cached HomeFeedDataJson directly, 200.
Cache missing or expired	GET /home	Aggregate a fresh payload from the Profile, FCE, Workout, Nutrition, and Progress services, upsert RecommendationCache, return the aggregated result, 200.
A downstream aggregation call fails	GET /home	Return 503 — SRV_SERVICE_UNAVAILABLE.

 **Backend Notes:** Tables: RecommendationCache (Read/Upsert). Cache-first edge-aggregator pattern.

Epic 8: Subscription & Billing Service (Lowest Priority)

User Story 8.1 — Get Subscription Status

Endpoint: GET /api/v1/subscription/status/{userId}

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
A UserSubscriptions row exists	GET /subscription/status/{userId}	Return Tier, Status, and ExpiresAt, 200.
No subscription row exists (defaults to Free)	GET /subscription/status/{userId}	Return 200 with Tier: 'Free'.

 *Backend Notes: Tables: UserSubscriptions (Read).*

User Story 8.2 — Upgrade to Premium

Endpoint: POST /api/v1/subscription/upgrade

Role: Authenticated

Request Payload

```
{ userId, planTier, durationMonths }
```

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Simulated billing authorization succeeds	POST /subscription/upgrade	Insert a BillingLogs row (PaymentStatus = Success), upsert UserSubscriptions to Tier = 'Premium' with a computed ExpiresAt, publish a subscription_upgraded event, return 200 with activeTier / expiresAt.
Simulated billing authorization fails	POST /subscription/upgrade	Insert a BillingLogs row (PaymentStatus = Failed), return 500 — SRV_DATABASE_ERROR.

 *Backend Notes: Tables: UserSubscriptions (Insert/Update), BillingLogs (Insert). Async Integration Hook: subscription_upgraded → Profile Service syncs IsPremiumCached = true; API Gateway lifts the rate limit to 2000 req/hr.*

User Story 8.3 — Cancel Subscription

Endpoint: POST /api/v1/subscription/cancel

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
An active Premium subscription exists	POST /subscription/cancel	Turn off the auto-renewal flag without truncating the current paid period, return 200.
No active subscription to cancel	POST /subscription/cancel	Return 404 — RES_NOT_FOUND.

 *Backend Notes: Tables: UserSubscriptions (Update). Cancellation takes effect at ExpiresAt, not immediately.*

Epic 9: Notification Service (Lowest Priority)

User Story 9.1 — Get Notifications

Endpoint: GET /api/v1/notifications

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
Notifications exist for the caller	GET /notifications	Return an unread-first list from InAppNotifications, 200.
Caller's tier has exceeded its rate-limit budget	GET /notifications	Return 429 — RATE_LIMIT_EXCEEDED.

 *Backend Notes: Tables: InAppNotifications (Read). Gateway tier limits: Anonymous 50/hr, Free 500/hr, Premium 2000/hr.*

User Story 9.2 — Mark Notification as Read

Endpoint: PUT /api/v1/notifications/{id}/read

Role: Authenticated

Acceptance Criteria

Given	When	Then (Expected Backend Behavior)
id exists and belongs to the caller	PUT /notifications/{id}/read	Set IsRead = true, return 200.
id not found	PUT /notifications/{id}/read	Return 404 — RES_NOT_FOUND.

 *Backend Notes: Tables: InAppNotifications (Update). The same table is written to by a background worker when broker events such as achievement_earned arrive.*

General Backend Requirements

Given	When	Then (Expected Backend Behavior)
All endpoints	Any request	Validate the JWT in the Authorization header (Bearer scheme). Return 401 — AUTH_TOKEN_INVALID if missing or malformed.
All responses	Any endpoint	Return the standard envelope: { isSuccess, message, data, errors, statusCode, timestamp }.
All paginated GET endpoints	GET list endpoints	Support ?page= and ?pageSize= (default 1 / 20). Return empty arrays instead of 404 when nothing matches.
Any request	Gateway rate limiting	Enforce tier-based caps at the API Gateway: Anonymous 50/hr, Free (Authenticated) 500/hr, Premium 2000/hr. Emit X-RateLimit-Limit / Remaining / Reset headers; return 429 — RATE_LIMIT_EXCEEDED when exceeded.
weight_updated / subscription_upgraded / achievement_earned events fire	Asynchronous pub/sub via message broker	Consuming services (FCE, Profile, Notification) must process events idempotently and never block the request that triggered them.
Smart Coach → FCE/Progress, Nutrition → FCE	Synchronous inter-service REST calls	Fail gracefully with 503 — SRV_SERVICE_UNAVAILABLE rather than hanging if a downstream service does not respond.

1. Authentication Service

Database Philosophy: This database is highly secure and specialized. It only stores information required to authenticate a user and manage security credentials. It does not store personal info like names or preferences.

Table: Users (Manages core identity)

Column Name	Type	Description / Example
Id	int	Primary Key (Identity)
Email	string	User's unique login email (Indexed, Unique, VarChar(255))
PasswordHash	string	Cryptographically hashed and salted password string (VarChar(512))
IsLockedOut	bool	Safety flag tracking brute-force states
LockedUntil	DateTime?	Expiration timestamp of the lockout penalty window (Nullable)
CreatedAt	DateTime	Record initialization timestamp

Table: RefreshTokens (Stores active tokens for sliding rotation sessions)

Column Name	Type	Description / Example
Id	int	Primary Key
UserId	int	Foreign Key linking directly to Users.Id
Token	string	Unique high-entropy token string (Unique Index, VarChar(512))
ExpiresAt	DateTime	When the token is no longer valid
CreatedAt	DateTime	Creation log timestamp
RevokedAt	DateTime?	Cancellation timestamp if explicitly invalidated (Nullable)

Table: OtpCodes (Stores temporary verification factors)

Column Name	Type	Description / Example
Id	int	Primary Key
Email	string	The email destination the code challenge was dispatched to (Indexed)
Code	string	The 6-digit hashed verification factor
ExpiresAt	DateTime	Strict expiration boundary (Exactly 10 minutes from creation)
IsUsed	bool	Single-use flag preventing replay vectors

Table: LoginAttempts (Audits consecutive authorization failures)

Column Name	Type	Description / Example
Id	int	Primary Key (Identity)
Email	string	Inbound email address submitted during authorization try (VarChar(255))
AttemptedAt	DateTime	High-precision event tracking timestamp
IsSuccess	bool	Boolean verification pass indicator
IpAddress	string	Client IP tracker variable (VarChar(45))

2. User Profile Service

Database Philosophy: This database stores all personal metadata and user-facing layout configurations. It links to the Authentication layer via a shared identifier (`UserId` used as primary identity).

Table: UserProfiles (Manages personal metadata)

Column Name	Type	Description / Example
Id	int	Primary Key (Must be the same as <code>Users.Id</code> from Auth database)
FirstName	string	User's first name, e.g., "Ahmed"
LastName	string	User's last name, e.g., "Mohamed"
Email	string	Denormalized replication for user display, e.g., "ahmed@example.com"
PhoneNumber	string	User contact phone number, e.g., "+201234567890"

Column Name	Type	Description / Example
ProfilePictureUrl	string	Location path string of image binary layout, e.g., "/images/profiles/1.jpg"
IsPremiumCached	bool	Local replica cache flag synced asynchronously from Subscription Microservice events
MemberSince	DateTime	Application profile creation milestone date

Table: UserSettings (Manages user localization preferences)

Column Name	Type	Description / Example
Id	int	Primary Key (Must match UserProfiles.Id cleanly)
Language	string	System localization code, e.g., "en"
Theme	string	Visual display configuration variable, e.g., "dark"
NotificationSettings	JSON/string	Toggles mapping tracking targets, e.g., { "workoutReminders": true, "mealReminders": true }
PrivacySettings	JSON/string	Visibility access map modifiers, e.g., { "profileVisibility": "private" }
UnitSettings	JSON/string	Metric configuration metrics constraints, e.g., { "weight": "kg", "height": "cm" }

3. Fitness Calculation Engine (FCE)

Database Philosophy: This represents the algorithmic "brains" of the system. It stores physical biometrics logs, holds target computed scientific variables, and runs matching configuration tables to output structural tracks.

Table: UserFitnessStats (Stores raw physical input variables)

Column Name	Type	Description / Example
Id	int	Primary Key (Must match Users.Id from Auth database)
Weight	double	Current recorded mass scale in kilograms, e.g., 75.5
Height	double	Current recorded vertical scaling configuration in centimeters, e.g., 175
Age	int	Current age parameter constraint, e.g., 25

Column Name	Type	Description / Example
Gender	string	Biological indicator checked during formulas execution, e.g., "Male"
Goal	string	Focus objective vector selected by the user, e.g., "Lose Weight"
ActivityLevel	string	Multiplier scale mapping, e.g., "Intermediate"
RecordedAt	DateTime	Explicit entry initialization timestamp

Table: CalculatedMetrics (Stores derived metabolic outputs)

Column Name	Type	Description / Example
Id	int	Primary Key (Must match Users.Id from Auth database)
BMR	double	Derived Basal Metabolic Rate score calculation, e.g., 1728.75
TDEE	double	Derived Total Daily Energy Expenditure value, e.g., 2679.56
CalorieTarget	double	Final system calculated target caloric allocation balance, e.g., 2179.56
Status	string	Evaluated classification tier baseline category, e.g., "Normal" (Weak / Normal / Hard)
CalculatedAt	DateTime	Explicit execution calculation completion timestamp

Table: FitnessPlanConfigs (Rulebook blueprint configurations)

Column Name	Type	Description / Example
Id	int	Primary Key identifier
Goal	string	Associated objective classifier pattern target, e.g., "Lose Weight"
Status	string	Associated calorie tier status constraint variable, e.g., "Normal"
CalorieMin	double	Minimum allocation threshold evaluation, e.g., 1801
CalorieMax	double	Maximum boundary constraints alignment, e.g., 2500
ExternalPlanId	string	Direct structural key pairing across external content libraries, e.g., "plan_lw_normal"
PlanName	string	Human-readable descriptor string, e.g., "Weight Loss - Normal Intensity"

Column Name	Type	Description / Example
WorkoutsPerWeek	int	Training day frequency per schedule, e.g., 4

Table: UserAssignedPlans (Tracks active strategy assignments)

Column Name	Type	Description / Example
Id	int	Primary Key (Must match <code>Users.Id</code> from Auth database)
ExternalPlanId	string	Connected strategy target config key identifier, e.g., "plan_lw_normal"
AssignedAt	DateTime	System strategy allocation timestamp
IsActive	bool	Execution tracker flag denoting current status

Table: UserPlanHistory (Immutable tracking reassignment audit logs)

Column Name	Type	Description / Example
Id	int	Primary Key (Identity)
UserId	int	Target user index identifier link
ExternalPlanId	string	Past active plan identifier record
AssignedAt	DateTime	Initial implementation timeline milestone
EndedAt	DateTime?	Termination tracking timestamp if recalculated (Nullable)
ReasonForChange	string	Action description context tracker, e.g., "weight_update"

4. Workout Service 💪

Database Philosophy: This represents the structural product content library. It houses blueprint templates for routine structures, exercise specifications arrays, and logs execution session initializations.

Table: WorkoutPlans (Top-level plan definitions)

Column Name	Type	Description / Example
Id	int	Primary Key

Column Name	Type	Description / Example
ExternalPlanId	string	Indexed, Unique mapping code paired directly across FCE logic, e.g., "plan_lw_normal"
Name	string	Title name string, e.g., "Weight Loss – Normal Intensity"
Description	string	Structural overview description block, e.g., "Balanced workout plan for weight loss"
Goal	string	Focus requirement string, e.g., "Lose Weight"
Status	string	Caloric tier mapping descriptor, e.g., "Normal"
Difficulty	string	Enforced scale classifier target, e.g., "Intermediate"

Table: Workouts (Specific training routine modules)

Column Name	Type	Description / Example
Id	int	Primary Key
WorkoutPlanId	int	Foreign Key mapping directly back to WorkoutPlans.Id
Name	string	Title layout label descriptor, e.g., "Full Body Strength"
Category	string	Index target tag classification grouping, e.g., "full-body"
Difficulty	string	Routine capability ranking target, e.g., "Intermediate"
DurationInMinutes	int	Target expected window scale allocation, e.g., 45
CaloriesBurn	int	Mathematically computed estimated output balance, e.g., 350
ImageUrl	string	Image location file path destination string, e.g., "/images/workouts/fb.jpg"
IsPremium	bool	Subscription gate flag protecting access paths

Table: Exercises (Master movement template definitions)

Column Name	Type	Description / Example
Id	int	Primary Key
Name	string	Clean movement name key, e.g., "Push-ups"
Description	string	Technical execution text instruction guidelines, e.g., "A calisthenic exercise..."
VideoUrl	string	Media storage loop file location string, e.g., "/videos/exercises/pushup.mp4"
TargetMuscles	JSON/string	String collection list arrays denoting targets, e.g., ["Chest", "Triceps"]

Column Name	Type	Description / Example
EquipmentNeeded	JSON/string	Structural gear requirement identifiers list, e.g., ["Bodyweight"]
Difficulty	string	Classification scale, e.g., "Beginner"

Table: WorkoutExercises (Template mapping sequence join table)

Column Name	Type	Description / Example
Id	int	Primary Key
WorkoutId	int	Foreign Key referencing parent routine identifier <code>Workouts.Id</code>
ExerciseId	int	Foreign Key referencing specific move index <code>Exercises.Id</code>
OrderIndex	int	Sequence positioning sorting metric configuration, e.g., 1
SetsDefault	int	Baseline training structure volume tracking variable, e.g., 3
RepsDefault	string	Default target iteration count layout format, e.g., "12-15"
RestTimeInSeconds	int	Target standard resting space constraint interval, e.g., 60

Table: WorkoutSessions (Isolated runtime track session instances)

Column Name	Type	Description / Example
SessionId	string	Globally unique random token tracking identifier (Primary Key)
UserId	int	Target user index identifier link (Indexed)
WorkoutId	int	Foreign Key matching target training reference row <code>Workouts.Id</code>
StartedAt	DateTime	Operational state runtime initiation timestamp
Status	string	Session management control status flag (Active , Completed , Abandoned)

5. Nutrition Service 🥗

Database Philosophy: This microservice acts as the programmatic repository for all recipe matrices, dietary structures, and caloric grouping frameworks.

Table: MealPlans (Target grouping frameworks configurations)

Column Name	Type	Description / Example
Id	int	Primary Key
Name	string	Menu categorization theme string, e.g., "High Protein 2200 kcal"
Description	string	Objective summary statement text block, e.g., "A plan for muscle gain."
CalorieTarget	int	Central energy property baseline target calculation index, e.g., 2200

Table: Meals (Specific culinary item profiles)

Column Name	Type	Description / Example
Id	int	Primary Key identifier
MealPlanId	int	Foreign Key matching back to MealPlans.Id
Name	string	Culinary label description name key, e.g., "Grilled Chicken Lunch"
MealType	string	Specific distribution slot allocation tier, e.g., "Lunch"
PrepTimeInMinutes	int	Operational preparation runtime limit target, e.g., 30
Difficulty	string	Execution capability rating marker, e.g., "Medium"
ImageUrl	string	Photo vector path asset directory pointer string, e.g., "/images/meals/1.jpg"
IngredientsJson	string	Raw text string details identifying absolute asset ratios
InstructionsJson	string	Text arrays detailing step-by-step preparation guidelines
VariationsJson	string?	Custom alternatives for adaptation paths (Nullable)
AllergensJson	string	Array data collection marking allergen inclusions, e.g., ["Dairy"]
TagsJson	string	Search meta classifiers array string configurations, e.g., ["Keto"]

Table: NutritionFacts (Granular nutritional density splits)

Column Name	Type	Description / Example
Id	int	Primary Key (1-to-1 relationship mapping directly to Meals.Id)
Calories	int	Target absolute caloric balance, e.g., 650

Column Name	Type	Description / Example
Protein	double	Macro distribution parameter mass scale indicator, e.g., 55.0
Carbs	double	Macro distribution parameter mass scale indicator, e.g., 60.0
Fats	double	Macro distribution parameter mass scale indicator, e.g., 20.0
Fiber	double	Minor macro classification parsing property metric, e.g., 8.0

Table: Ingredients (Master physical asset items library)

Column Name	Type	Description / Example
Id	int	Primary Key
Name	string	Common taxonomy text element label, e.g., "Chicken Breast"

Table: MealIngredients (Join bridge linking ratios to menus)

Column Name	Type	Description / Example
Id	int	Primary Key
MealId	int	Foreign Key mapping to the target container row Meals.Id
IngredientId	int	Foreign Key linking the physical item Ingredients.Id
Amount	string	Quantity dimension descriptor layout value, e.g., "150g"

Table: MealPlanItems (Weekly calendar scheduling configurations)

Column Name	Type	Description / Example
Id	int	Primary Key
MealPlanId	int	Foreign Key mapping structural connections down to MealPlans.Id
MealId	int	Foreign Key linking specific components index Meals.Id
DayOfWeek	string	Scheduling placement marker constraints, e.g., "Monday"
MealTime	string	Target distribution window designation index, e.g., "Breakfast"

6. Smart Coach Service 🤖

Database Philosophy: Governs user dialog records and dashboard rendering data caches. It is highly transactional and supports localized document optimizations.

Table: ChatSessions (Dialogue grouping threads)

Column Name	Type	Description / Example
Id	int	Primary Key
UserId	int	Target account user reference link (Indexed Foreign Key)
StartedAt	DateTime	Initial implementation timeline milestone date
Title	string	Custom conversation categorization layout identifier, e.g., "Keto Planning"

Table: ChatMessages (Archives individual conversation line entries)

Column Name	Type	Description / Example
Id	int	Primary Key
ChatSessionId	int	Foreign Key tracking back directly to ChatSessions.Id
Role	string	Actor label context constraint value, e.g., "user" or "assistant"
Content	string	Raw dialogue text payload copy block
Timestamp	DateTime	Event persistence creation log time

Table: RecommendationCache (High-performance home screen dashboard caches)

Column Name	Type	Description / Example
UserId	int	System user primary identifier reference (Primary Key)
UserContextJson	string	Text representation mapping physical metadata snapshot states
HomeFeedDataJson	string	Full serialized data tree caching comprehensive dashboard lookups
CachedAt	DateTime	Real-time cache row compilation initialization timestamp
ExpiresAt	DateTime	Explicit expiration boundary (Standard 30-minute window limit)

7. Progress Tracking Service

Database Philosophy: This represents the write-heavy performance analytics engine. It functions as the permanent database tracking user activity metrics over time.

Table: WorkoutLogs (Routines completion metrics summaries)

Column Name	Type	Description / Example
Id	int	Primary Key
UserId	int	Target tracking actor identifier (Indexed Foreign Key)
WorkoutId	int	Unconnected template identifier mapping back to Workout service parameters
SessionId	string	Unique token tracker mapping to the runtime initialization instance
DurationInMinutes	int	Actual runtime spent by user executing the routine, e.g., 45
CaloriesBurned	int	Calculated energy metrics expenditure score, e.g., 350
Rating	int	Boundary verified structural score configuration, e.g., 5 (Scale 1–5)
Notes	string	User narrative input feedback data copy block (Nullable)
CompletedAt	DateTime	Processing execution end timestamp log

Table: WorkoutLogExercises (Set-by-set granular volume trackers)

Column Name	Type	Description / Example
Id	int	Primary Key (Identity)
WorkoutLogId	int	Foreign Key tracking back directly to the parent layout WorkoutLogs.Id
ExerciseId	int	Unconnected movement template index identifier
SetsCompleted	int	Actual volume workload configuration tracking variables, e.g., 3
RepsCompleted	int	Actual iteration tracking variable completed, e.g., 12
WeightUsed	double	Resistance mass configuration used under tracking operations, e.g., 80.0
Completed	bool	Validation state flag confirming execution success loops

Table: WeightHistory (Chronological mass check logs)

Column Name	Type	Description / Example
Id	int	Primary Key
UserId	int	Target user index reference link (Indexed Foreign Key)
Weight	double	Numeric mass parameter indicator scale input value, e.g., 72.5
Date	DateTime	Day calendar timeline event registration point
Notes	string	Short description tracking note parameter (Nullable)

Table: BodyMeasurements (Physical perimeter scaling trackers)

Column Name	Type	Description / Example
Id	int	Primary Key (Identity)
UserId	int	Target system user metric tracking link (Indexed)
Neck / Chest / Biceps / Waist / Hips / Thighs	double?	Dimensional metrics track inputs (Nullable strings for open tracking parameters)
RecordedAt	DateTime	Calendar tracking timeline event marking

Table: Achievements (Master milestone system rewards definitions)

Column Name	Type	Description / Example
Id	int	Primary Key
Name	string	Reward label text title string, e.g., "7-Day Streak"
Description	string	Qualification requirements summary copy block, e.g., "Worked out for 7 days..."
IconUrl	string	Image location file string reference, e.g., "/images/badges/streak7.png"

Table: UserAchievements (Join bridge tracking unlocked awards)

Column Name	Type	Description / Example
Id	int	Primary Key

Column Name	Type	Description / Example
UserId	int	Target account user reference link (Indexed Foreign Key)
AchievementId	int	Foreign Key mapping directly back to <code>Achievements.Id</code>
EarnedAt	DateTime	Realization milestone timeline tracking event

Table: Streaks (Monitors execution consistency loops)

Column Name	Type	Description / Example
UserId	int	Shared tracking profile key anchor (Primary Key)
CurrentStreak	int	Current consecutive calendar tracking interval counters accumulation, e.g., 8
LongestStreak	int	High-water mark consistency metric tracker preservation score, e.g., 21
LastWorkoutDate	DateTime?	Calendar reference used to analyze gap thresholds (Nullable)

Table: UserStatistics (High-speed cached query summaries repository)

Column Name	Type	Description / Example
Id	int	Primary Key (Must match <code>Users.Id</code> from Auth database)
TotalWorkouts	int	Aggregated cumulative metric count indicator index value, e.g., 46
TotalCaloriesBurned	int	Aggregated cumulative metric count indicator index value, e.g., 12850
CurrentWeight	double	Synced replication from newest active mass entries tracking, e.g., 72.5
StartWeight	double	Baseline profile capture weight entry parameter record tracking, e.g., 75.5
TotalWeightLost	double	Mathematically computed net progress scale tracking metric variance
UpdatedAt	DateTime	Profile row modification state processing timestamp

8. Subscription & Billing Service

Database Philosophy: The explicit system source of truth governing financial status flags, access tier authorizations, transactional metrics histories, and limit gate permissions.

Table: UserSubscriptions (Membership authorization tracking profiles)

Column Name	Type	Description / Example
Id	int	Primary Key (Identity)
UserId	int	Context account owner reference tracker link (Unique Index)
Tier	string	Active classification boundary parameter choice (Free , Premium)
Status	string	System capability processing control state flag (Active , Expired , Cancelled)
StartsAt	DateTime	Initial execution timeline deployment date
ExpiresAt	DateTime	Absolute termination processing deadline checkpoint
UpdatedAt	DateTime	Record transaction index entry modification event

Table: BillingLogs (Audit payment receipt files logs)

Column Name	Type	Description / Example
Id	int	Primary Key (Identity)
UserId	int	Associated user identity indexing criteria marker (Indexed)
ReferenceId	string	External mock transaction checkout confirmation unique code (VarChar(100))
Amount	decimal	Financial billing ledger transaction metric formatting data scale (199.99)
Currency	string	ISO localization financial ticker definition, e.g., "EGP"
PaymentStatus	string	Log confirmation status execution results values (Success , Failed)
Timestamp	DateTime	Transaction logging processing milestone

9. Notification Service

Database Philosophy: Functions as a decoupled tracking cache system archiving alert records and routing parameters for inside-app messaging lists.

Table: InAppNotifications (App layout messaging feeds entries logs)

Column Name	Type	Description / Example
Id	int	Primary Key (Identity)
UserId	int	Target notification distribution destination identifier (Indexed)
Title	string	Notification heading copy block text identifier string, e.g., "7-Day Streak!"
Message	string	Detailed content instruction narrative copy string, e.g., "Worked out 7 consecutive days..."
Type	string	Category tracking identifier flags mapping (AchievementAlert , StreakWarning)
IsRead	bool	Boolean interaction check tracking state flag (Defaults to false)
SentAt	DateTime	System delivery network outbound timestamp logging